课程提纲

- 机器学习概述
- 机器学习线性模型（回归/分类）
- 支持向量机与核方法
- 神经网络与深度学习
- 学习机理与优化算法
- 数据降维与聚类
- 隐变量模型与表征学习
- 概率图模型
- 前沿应用

Neural Networks

Liqing Zhang & Shikui Tu
Shanghai Jiao Tong University

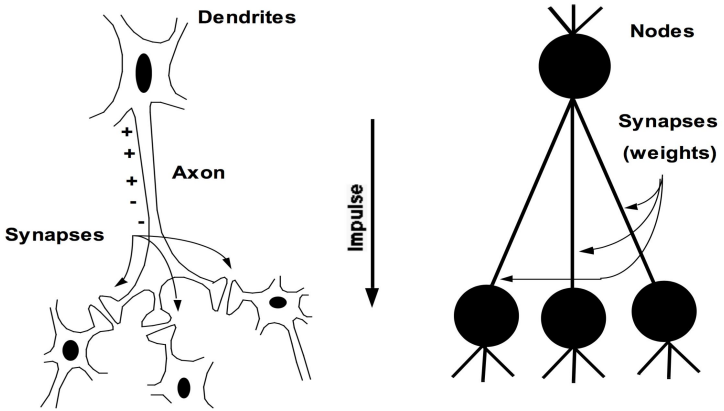
Contents

- Biological Background
- Single Layer Neural Model
- Multilayer Perception Model
  - Backpropgation algorithm
- Model Selection Problem
- Training Issues
  - Learning acceleration
  - Regularization (Constraint, Sparse, Dropout, Early-stop)
  - Data Augumentation
- Advantages and Disanvantages

Connectionist Models

- Consider humans:
  - Neuron switching time： ~ 0.001 second
  - Number of neurons： ~ 10^{10}
  - Connections per neuron： ~ $10^{4\text{-}5}$
  - Scene recognition time： ~ 0.1 second
  - 100 inference steps doesn't seem like enough

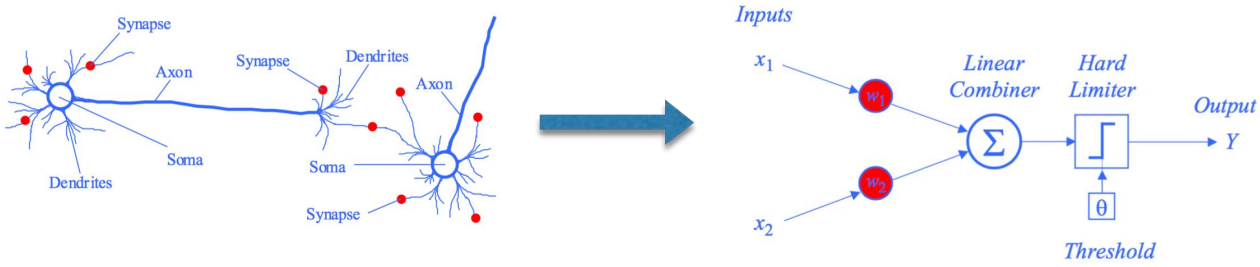
→much parallel computation

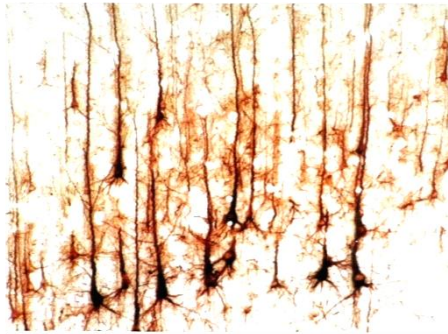


- Properties of artificial neural nets (ANN)
  - Many neuron-like threshold switching units
  - Many weighted interconnections among units
  - Highly parallel, distributed processes

Perceptron and Neural Nets

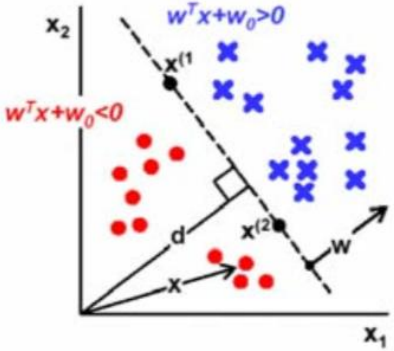
- From biological neuron to artificial neuron (perceptron)



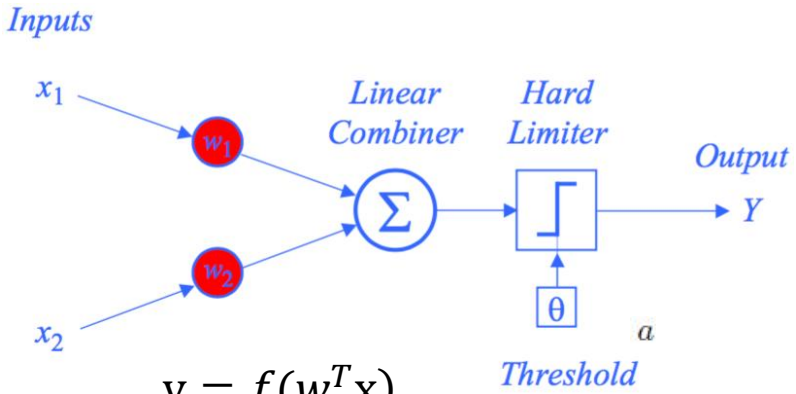


- Model formulation

$$s = w^T x + w_0 = \sum_{j=1}^{n} w_j x_j + w_0, \qquad y = \begin{cases} +1, & \text{if } s > 0; \\ -1, & \text{if } s < 0 \end{cases}$$



Perceptrons



$$y = f(w^T x)$$

$$f(a) = \begin{cases} +1, & a \geqslant 0 \\ -1, & a < 0. \end{cases}$$

Error function minimization

The training data: $\{(x_n, y_n)\}_{n=1}^{N} = \{(x_1, y_1), (x_2, y_2), \cdots, (x_N, y_N)\}$

The perceptron criterion: $E(w) = -\sum_{n=1}^{N} w^T x_n y_n$

where N denotes the set of all misclassified patterns

Weight Updating Rule

- Apply the stochastic gradient descent algorithm to this error function

$$w^{t+1} = w^t - \eta \nabla E(w) = w^t + \eta x_n y_n$$

- The contribution to the error from a misclassified pattern will be reduced because

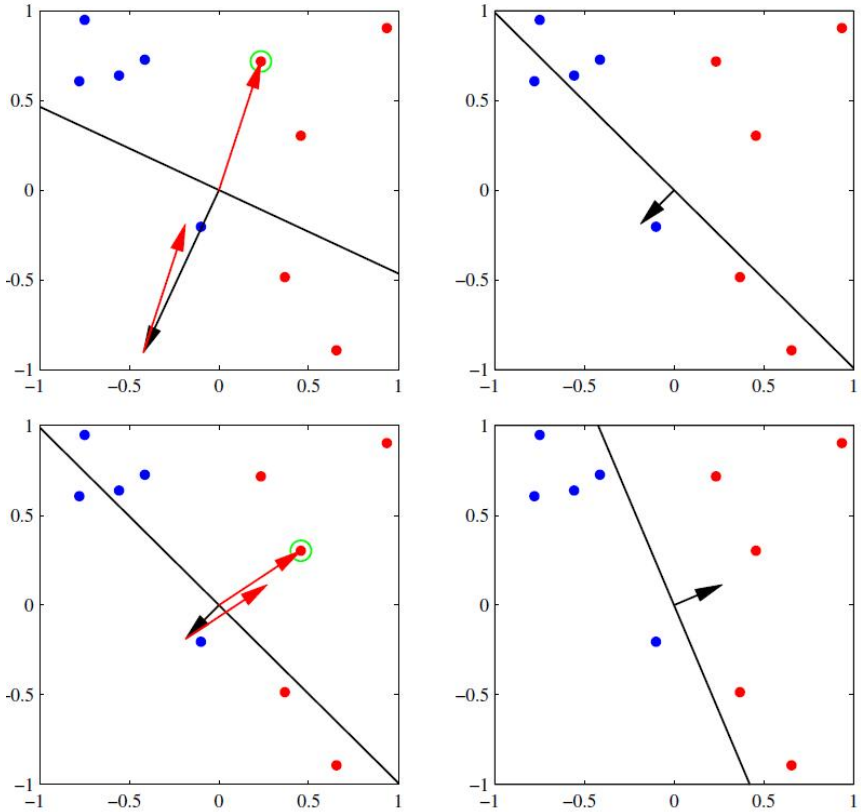
$$E_n(W^{t+1}) = -\left(w^{t+1}\right)^T x_n y_n = -\left(w^t + \eta x_n y_n\right)^T x_n y_n$$

$$= -\left(w^t\right)^T x_n y_n - \eta x_n^T x_n y_n y_n \le -\left(w^t\right)^T x_n y_n = E_n(W^t)$$

The perceptron convergence theorem:

- If the training data set is linearly separable, then the perceptron learning algorithm is guaranteed to find an exact solution in a finite number of steps.
- For data sets that are not linearly separable, the perceptron learning algorithm will never converge.

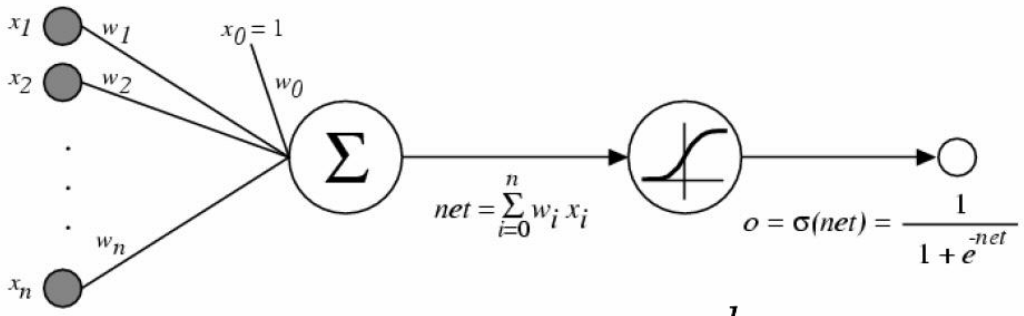
An illustration



$$w^{t+1} = w^t - \eta \nabla E(w) = w^t + \eta x_n y_n$$

From PRML book Fig. 4.7

The learning algorithm



- Recall the nice property of sigmoid function $\frac{d\sigma}{dt} = \sigma(1-\sigma)$
- Consider regression problem $f: X \rightarrow Y$, for scalar Y: $y = f(x) + \epsilon$
- Let's maximize the conditional data likelihood

$$\vec{w} = \arg\max_{\vec{w}} \ln \prod_i P(y_i | x_i; \vec{w})$$

$$\vec{w} = \arg\min_{\vec{w}} \sum_i \frac{1}{2}(y_i - \hat{f}(x_i; \vec{w}))^2$$

Gradient Descent

$$\frac{\partial E(w)}{\partial w_i} = \frac{\partial}{\partial w_i}\left(\frac{1}{2}\sum_d (y_d - o_d)^2\right)$$

$$= \left(\frac{1}{2}\sum_d 2(y_d - o_d)\frac{\partial}{\partial w_i}(y_d - o_d)\right)$$

$$= -\sum_d (y_d - o_d)\left(\frac{\partial o_d}{\partial net}\frac{\partial net}{\partial w_i}\right)$$

$$= -\sum_d (y_d - o_d)\, o_d\,(1 - o_d)\, x_d^i$$

x_d = input

y_d = target output

o_d =observed unit output

w_i =weigh i

$$net = w^T x,$$

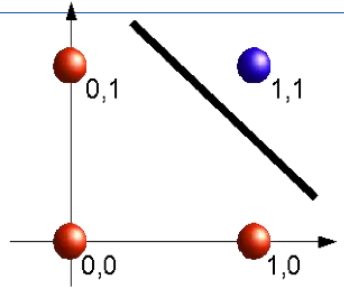
$$o_d = f(net)$$

Do until converge:

1. compute gradient $\nabla \mathbf{E}_D[\mathbf{w}]$

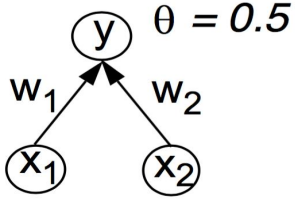
$$\vec{w} = \vec{w} - \eta \nabla E_D[\vec{w}]$$

Limitations for Perceptron



NAND

X	Y	Z(color)
0	0	1
0	1	1
1	0	1
1	1	0



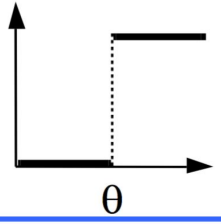
$\theta = 0.5$

$$y = f\left(w^T x\right) = f\left(w_1 x_1 + w_2 x_2\right)$$

$$x^{(1)} = (0,0); \quad x^{(2)} = (0,1);$$

$$x^{(3)} = (1,1); \quad x^{(4)} = (1,0);$$

$$y = \{1,0,1,0\}$$

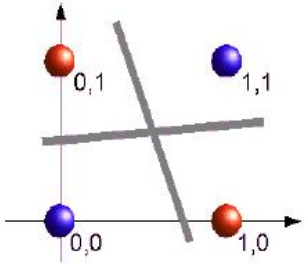


$$f(a) = \begin{cases} 1, \text{ for } a > \theta \\ 0, \text{ for } a \le \theta \end{cases}$$

some possible values for w_1 and w_2

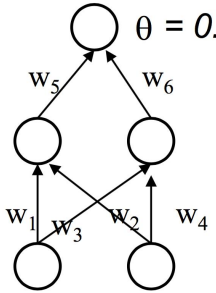
w_1	w_2
0.20	0.35
0.20	0.40
0.25	0.30
0.40	0.20

Limitations for Perceptron

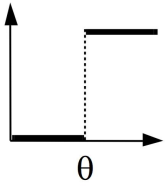


XOR

x	y	Z (color)
0	0	0
0	1	1
1	0	1
1	1	0



$\theta = 0.5$ *for all units*



$$f(a) = \begin{cases} 1, \text{ for } a > \theta \\ 0, \text{ for } a \leq \theta \end{cases}$$

A possible set of values for $(w_1, w_2, w_3, w_4, w_5, w_6)$: (0.6,-0.6,-0.7,0.8,1,1)

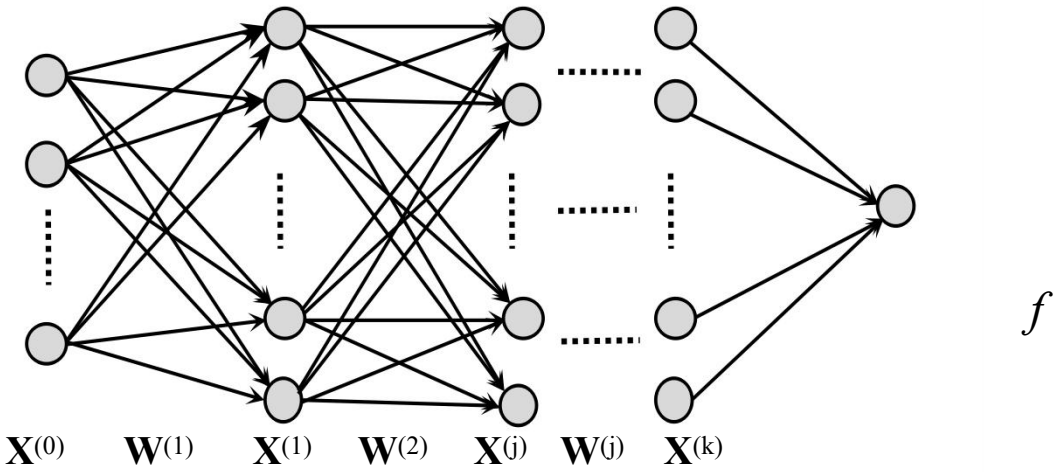
Contents

- Biological Background
- Single Layer Neural Model
- Multilayer Perception Model
  - Backpropgation algorithm
- Model Selection Problem
- Training Issues
  - Learning acceleration
  - Regularization (Constraint, Sparse, Dropout, Early-stop)
  - Data Augumentation
- Advantages and Disanvantages

Notation

- Hidden unit $\mathbf{X}^{(j)}$: neurons in all but the last layer
- Output of j-th layer: $\mathbf{X}^{(j)} \rightarrow$ input of $(j+1)$-th layer
- Input vector: $\mathbf{X}^{(0)}$ Final output: f
- The weight of i-th sigmoid unit in the j-th layer: $\mathbf{W}_i^{(j)}$
- Number of sigmoid units in j-th layer: m_j

$$s_i^{(j)} = \mathbf{X}^{(j-1)} \cdot \mathbf{W}_i^{(j)}$$



The Backpropagation Algorithm

- Gradient of $\mathbf{W}_i^{(j)}$:

$$s_i^{(j)} = \mathbf{X}^{(j-1)} \cdot \mathbf{W}_i^{(j)}$$

$$\frac{\partial \varepsilon}{\partial \mathbf{W}_i^{(j)}} = \frac{\partial \varepsilon}{\partial s_i^{(j)}} \frac{\partial s_i^{(j)}}{\partial \mathbf{W}_i^{(j)}} = \frac{\partial \varepsilon}{\partial s_i^{(j)}} \mathbf{X}^{(j-1)}$$

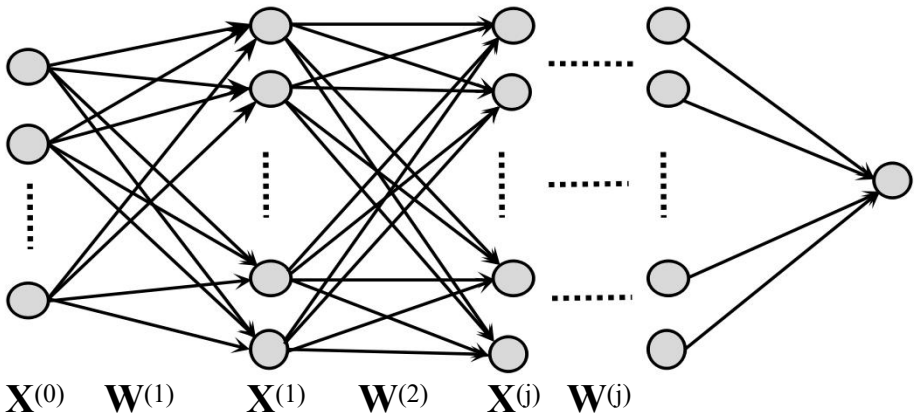
$$\frac{\partial \varepsilon}{\partial s_i^{(j)}} = \frac{\partial (d-f)^2}{\partial s_i^{(j)}} = -2(d-f)\frac{\partial f}{\partial s_i^{(j)}}$$

$$= -2(d-f)\frac{\partial f}{\partial s_i^{(j)}} \mathbf{X}^{(j-1)} = -2\delta_i^{(j)} \mathbf{X}^{(j-1)}$$

Local gradient

- Weight update:

$$\mathbf{W}_i^{(j)} \leftarrow \mathbf{W}_i^{(j)} + \eta \delta_i^{(j)} \mathbf{X}^{(j-1)}$$



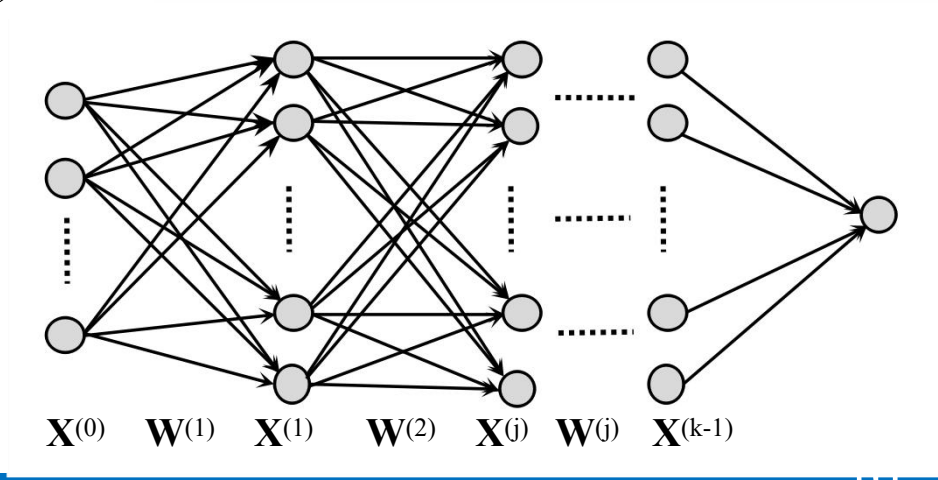
Weight Changes in the Final Layer

- Local gradient:

$$\delta^{(k)} = (d - f)\frac{\partial f}{\partial s_i^{(j)}}$$
$$= (d - f) f (1 - f)$$

- Weight update:

$$\mathbf{W}^{(k)} \leftarrow \mathbf{W}^{(k)} + \eta (d - f) f (1 - f) \mathbf{X}^{(k-1)}$$



Weights in Intermediate Layers

- Local gradient:

$$\delta_i^{(j)} = (d-f)\frac{\partial f}{\partial s_i^{(j)}}$$

$$= (d-f)\left[\frac{\partial f}{\partial s_1^{(j+1)}}\frac{\partial s_1^{(j+1)}}{\partial s_i^{(j)}} + \ldots \frac{\partial f}{\partial s_l^{(j+1)}}\frac{\partial s_l^{(j+1)}}{\partial s_i^{(j)}} + \ldots + \frac{\partial f}{\partial s_{m_{j+1}}^{(j+1)}}\frac{\partial s_{m_{j+1}}^{(j+1)}}{\partial s_i^{(j)}}\right]$$

$$= \sum_{l=1}^{m_{j+1}} (d-f)\frac{\partial f}{\partial s_l^{(j+1)}}\frac{\partial s_l^{(j+1)}}{\partial s_i^{(j)}} = \sum_{l=1}^{m_{j+1}} \delta_l^{(j+1)}\frac{\partial s_l^{(j+1)}}{\partial s_i^{(j)}}$$

  - The final ouput f, depends on $s_i^{(j)}$ through of the summed inputs to the sigmoids in the $(j+1)$-th layer.

- Need for computation of $\dfrac{\partial s_l^{(j+1)}}{\partial s_i^{(j)}}$

Weight Update in Hidden Layers (cont.)

$$s_l^{(j+1)} = \mathbf{X}^{(j)} \cdot \mathbf{W}_l^{(j+1)} = \sum_{v=1}^{m_j+1} f_v^{(j)} w_{v,l}^{(j+1)}$$

$$\frac{\partial s_l^{(j+1)}}{\partial s_i^{(j)}} = \frac{\partial \left[\sum_{v=1}^{m_j+1} f_v^{(j)} w_{v,l}^{(j+1)} \right]}{\partial s_i^{(j)}} = \sum_{v=1}^{m_j+1} w_{v,l}^{(j+1)} \frac{\partial f_v^{(j)}}{\partial s_i^{(j)}}$$

$$= w_{i,l}^{(j+1)} f_i^{(j)} (1 - f_i^{(j)})$$

$$s_i^{(j)} = \mathbf{X}^{(j-1)} \cdot \mathbf{W}_i^{(j)}$$

- $v \neq i$: $\dfrac{\partial f_v^{(j)}}{\partial s_i^{(j)}} = 0$ $\quad$ $v = i$: $\dfrac{\partial f_v^{(j)}}{\partial s_i^{(j)}} = f_v^{(j)} (1 - f_v^{(j)})$

- Conseqeuntly,

$$\delta_i^{(j)} = f_i^{(j)} (1 - f_i^{(j)}) \sum_{l=1}^{m_j+1} \delta_l^{(j+1)} w_{i.l}^{(j+1)}$$

Weight Update in Hidden Layers (cont.)

- Attention to **recursive equation** of local gradient!

$$\delta^{(k)} = f(1-f)(d-f)$$

$$\delta_i^{(j)} = f_i^{(j)}(1-f_i^{(j)})\sum_{l=1}^{m_j+1}\delta_i^{(j+1)}w_{i.l}^{(j+1)}$$

- Backpropagation:
  - Error is back-propagated from the output layer to the input layer
  - Local gradient of the latter layer is used in calculating local gradient of the former layer.

3.3.5 (cont.)

- Example (even parity function)

$$f = x_1 x_2 + \bar{x}_1 \bar{x}_2$$

input			target
1	0	1	0
0	0	1	1
0	1	1	0
1	1	1	1

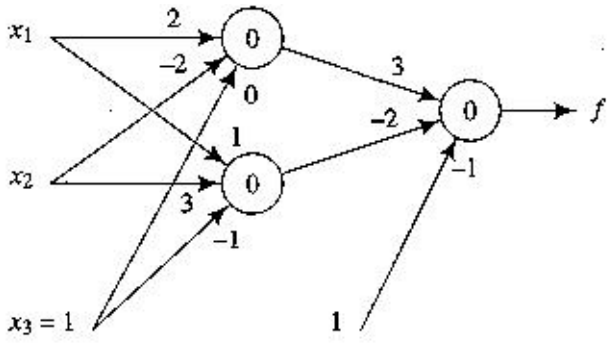


Figure 3.6
A Network to Be Trained by Backprop

  - Learning rate: 1.0

Generalization, Accuracy, Overfitting

- **Generalization ability:**
  - NN appropriately classifies vectors not in the training set.
  - Measurement = accuracy
- **Curve fitting**
  - Number of training input vectors ≥ number of degrees of freedom of the network.
  - In the case of *m* data points, is (*m*-1)-degree polynomial best model? **No, it can not capture any special information.**
- **Overfitting**
  - Extra degrees of freedom are essentially just fitting the noise.
  - Given sufficient data, the *Occam's Razor* principle dictates to choose the lowest-degree polynomial that adequately fits the data.

Overfitting

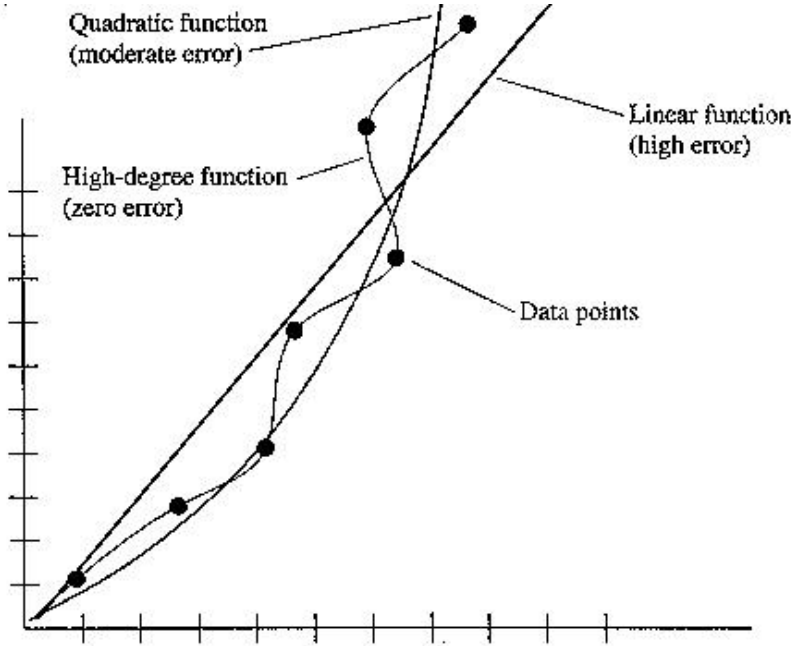


Figure 3.7

Curve Fitting

Contents

- Biological Background
- Single Layer Neural Model
- Multilayer Perception Model
  - Backpropgation algorithm
- Training Issues
  - Learning acceleration
  - Regularization (Constraint, Sparse, Dropout, Early-stop)
  - Data Augumentation
- Advantages and Disanvantages

Model Selection Problem

- **Best NN Architecture**
  - The number of layers
  - The number of neurons in each layer
  - The activation function
- **Regularization**
  - Sparse Constraint
  - Shrinkage Constraint
  - Disentagled Representation

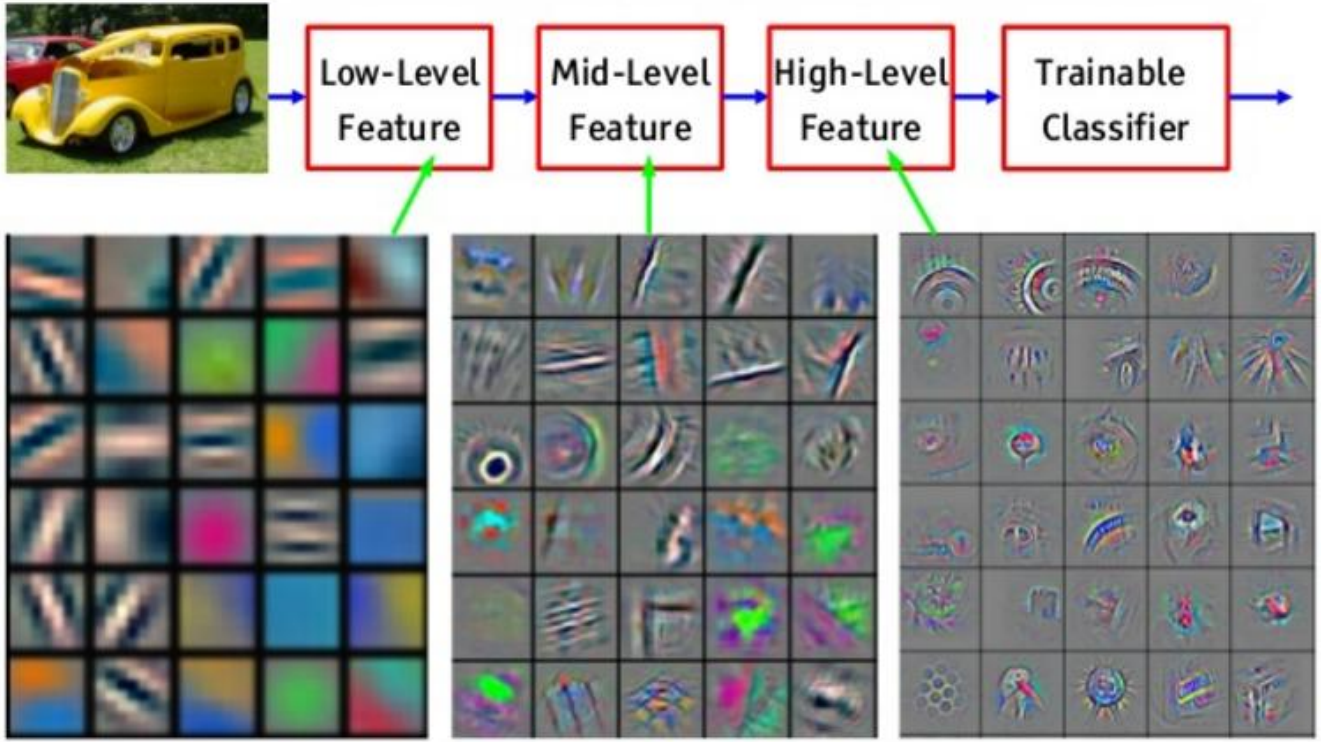
Training Issues

- Gradient descent for weight vector of entire network
- Local minimum problem: Gradient descent may converge to a local, not necessarily global error minimum
  - No theoretical guarantee due to highly nonlinearity
  - Practically, Genetic algorithm; Random initialization; Regularization; Dropout
- Speed up learning procedure: weight momentum α

$$\Delta w_{i,j}(t) = \eta \delta_j x_{i,j} + \alpha \Delta w_{i,j}(t-1)$$

- Generalization Problem:
  - Cross validation; Regularization; Dropout; Pretrained Model

Hierarchical feature representations

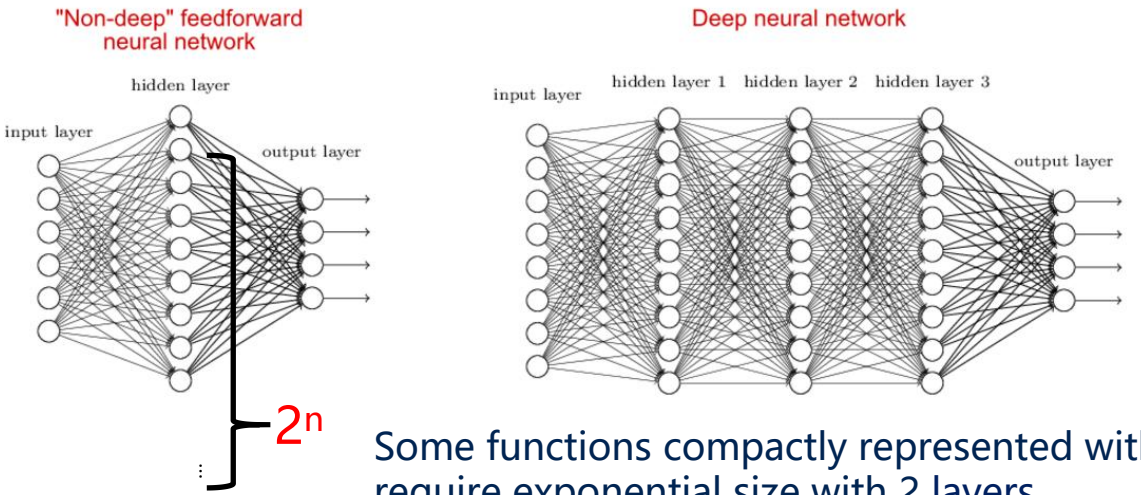


(Figure from NIPS' 2015 Tutorial by Geoff Hinton, Yoshua Bengio & Yann LeCun)

Universal approximation theorem

Any continuous function f $f : R^N \rightarrow R^M$

can be realized by a network with one hidden layer (given enough hidden neurons) if the activation function is not polynomial.



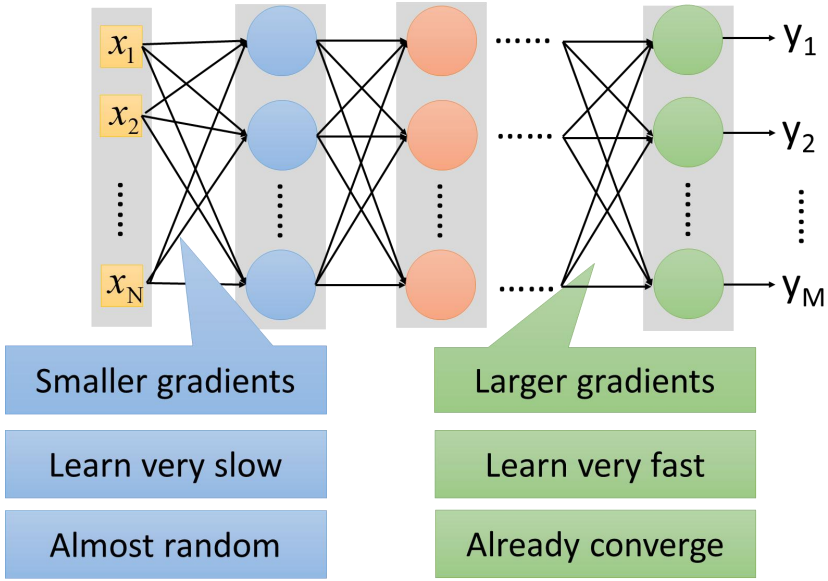
Some functions compactly represented with k layers may require exponential size with 2 layers

Vanishing gradient

Recall BP algorithm:

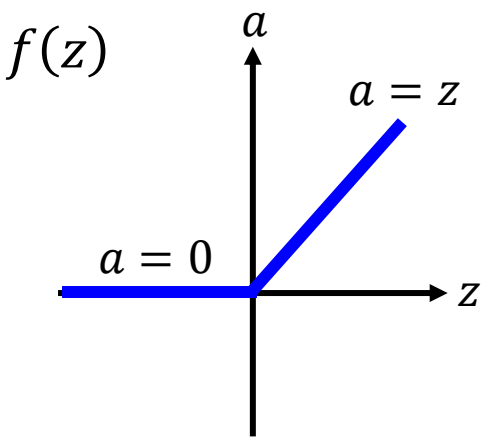
For each hidden unit *h*

$$\delta_h = o_h^{(1)}(1 - o_h^{(1)}) \sum_{k \in outputs} w_{h,k}\delta_k$$



Solution to Vanishing Gradient

- Rectified Linear Unit (ReLU)
- No vanishing gradients



$$f'(z) = \begin{cases} 1, & if\ z > 0 \\ 0, & otherwise \end{cases}$$

$$\delta_h = f'(net_h) \sum_{k \in outputs} w_{h,k} \delta_k$$

Neuron design

- Hidden layer: ReLU and variants of ReLU
- Output neuron:
  - Binary classification: sigmoid
  - Multi-class problem: softmax
  - Regression or parameter estimation: linear (e.g. linear units for Gaussian means)

Loss function

- Maximum likelihood principle (the cost function of most NNs)

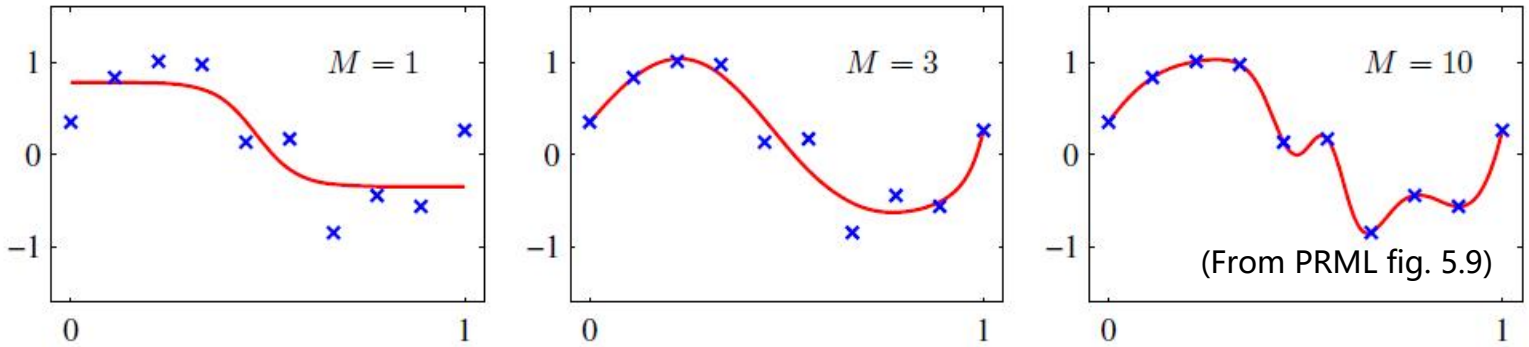
$$\hat{\Theta} = \arg\min \sum_{k=1}^{N} -\log p(y_k \mid x_k, \Theta)$$

- Work with sigmoid/softmax units

- Mean squared error (MSE); Cross Entropy for Classification
- Work with linear units

Regularization in Neural Networks

- The number of input and output units in a neural network is generally determined by the dimensionality of the data set
  - The number M of hidden units is a free parameter that can be adjusted to give the best predictive performance



(From PRML fig. 5.9)

Weight Shrinkage

- Choose a relatively large value for M and then to control complexity by the addition of a regularization term to the error function. The simplest regularizer is the quadratic, giving a regularized error of the form

$$\widetilde{E}(W) = E(W) + \frac{\lambda}{2} W^T W$$

- The effective model complexity is then determined by the choice of the regularization coefficient λ.
- The regularizer can be interpreted as the negative logarithm of a zero-mean Gaussian prior distribution over the weight vector **w**.
- Limitations: Inconsistent with certain scaling properties of network mappings

Invariances

- Predictions should be unchanged, or *invariant*, under one or more transformations of the input variables.
- Encourage model to exhibit the required invariances
  - **Data Augumentation:** The **training set is augmented** using replicas of the training patterns, transformed according to the desired invariances
  - A regularization term is added to the error function that penalizes changes in the model output when the input is transformed. This leads to the technique of *tangent propagation*.
  - Invariance is built into the pre-processing by **extracting features that are invariant** under the required transformations. Any subsequent regression or classification system that uses such features as inputs will necessarily also respect these invariances.
  - The final option is to **build the invariance properties into the structure of a neural network**

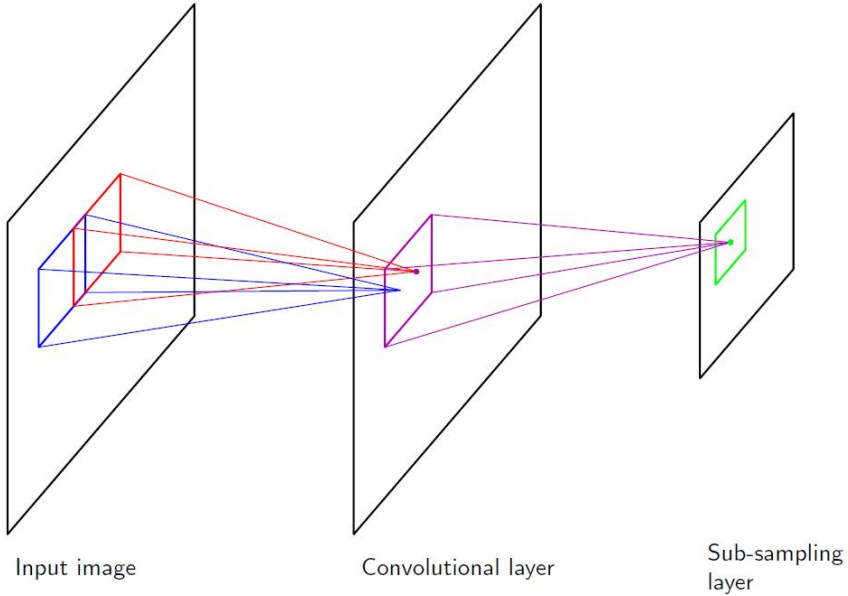
Convolutional networks

Another approach to creating models that are invariant to certain transformation of the inputs is to build the invariance properties into the structure of a neural network. This is the basis for the convolutional neural network.

(i) local receptive fields,

(ii)weight sharing, and

(iii)subsampling.



THE END OF TALK

讨论题

- 神经网络模型与之前所学模型的联系与区别
- 什么是模型训练的过拟合问题？产生的原因是什么？有什么处理方法？
- 什么是模型的正则化？模型正则化有哪些方法？
- 为什么现在的神经网络模型倾向于使用深而窄的结构，而不是浅而宽的结构？（注：“深”指层数多，“窄”指每层的节点数少）
- 神经网络模型隐藏层和输出层节点常用的激活函数是什么，损失函数是什么？为什么这样选择？